

Projet : RP

CHAMBOLLE-SOLAZ Aurélien

MAZLUM Jules

1 Introduction

La planification d'itinéraires en ligne est un enjeu central dans de nombreux domaines. Les systèmes actuels prennent souvent en compte des informations statiques comme les distances et les limitations de vitesse, mais en pratique, des événements imprévus tels que des accidents ou des intempéries peuvent perturber les itinéraires. Cela justifie le besoin d'algorithmes capables de s'adapter dynamiquement à l'environnement.

Ce projet s'inscrit dans cette problématique en s'intéressant au *Canadian Traveller Problem* (CTP), un problème combinatoire où certaines routes d'un graphe peuvent être bloquées et où ces blocages ne sont découverts qu'en temps réel. Nous nous concentrons ici sur une version étendue, appelée *Covering Canadian Traveller Problem* (CCTP), où l'objectif est de visiter tous les sommets d'un graphe complet malgré des blocages dynamiques, puis de revenir au sommet de départ.

Deux algorithmes sont proposés et comparés expérimentalement : une approche cyclique (**CR**) et une autre mêlant heuristique de Christophides et plus proche voisin (**CNN**).

2 Problématique

On considère un graphe complet non orienté $G = (V, E)$, pondéré par des coûts satisfaisant l'inégalité triangulaire. Un sommet de départ $s \in V$ est fixé. Le voyageur connaît la structure du graphe, mais certaines arêtes peuvent être bloquées. Ces blocages ne sont révélés qu'en atteignant un sommet adjacent à l'arête.

Le nombre de blocages est borné par un entier $k < |V| - 1$, et le graphe reste connexe malgré ceux-ci. L'objectif est de trouver une tournée partant de s , visitant tous les

sommets puis revenant à s , tout en minimisant le coût réel du trajet, malgré les blocages découverts dynamiquement.

Ce problème est une variante dynamique du *Travelling Salesman Problem* (TSP), enrichie d'incertitudes. La performance des algorithmes est mesurée via le *rapport de compétitivité*, comparant le coût réel à celui du chemin optimal hors-ligne (avec connaissance parfaite des blocages).

3 Algorithmes étudiés

3.1 CR : Routage cyclique

Principe général

L'algorithme **CR** repose sur une stratégie itérative. On commence par générer un tour initial P avec l'algorithme de Christophides. Ce tour sert d'ordre de visite prioritaire.

Le voyageur tente de suivre ce tour dans un sens donné. En cas de blocage, il tente un raccourci. Si aucun contournement n'est possible, on inverse le sens et recommence.

Étapes de l'algorithme :

- Génération du tour avec **Christophides**(graphe, depart)
- Parcours du tour en tenant compte des blocages
- Recherche d'alternatives en cas d'obstacle
- Inversion du sens si nécessaire
- Concaténation des sous-tours pour construire le parcours complet

L'algorithme :

Entrée :

G : graphe complet (V, E) avec arêtes possiblement bloquées

s : sommet de départ

Sortie : Un parcours visitant tous les sommets malgré les blocages

1. $P \leftarrow$ tour obtenu avec l'algorithme de Christofides sur G
2. Supprimer le dernier sommet de P si c'est s (pour éviter doublon)
3. Initialiser :
 - $\text{non_visités} \leftarrow V \setminus \{s\}$
 - $\text{parcours} \leftarrow [s]$
 - $\text{arêtes_bloquées} \leftarrow \emptyset$
 - $\text{sens} \leftarrow 1$ (horaire)
 - $\text{courant} \leftarrow s$
4. Tant que non_visités n'est pas vide :
 - a. Débuter une nouvelle itération depuis courant
 - b. $\text{visited_ité} \leftarrow \emptyset$
 - c. Répéter :
 - i. $\text{next} \leftarrow$ sommet suivant dans P selon sens
 - ii. Si next est déjà visité : passer au suivant
 - iii. Si l'arête $(\text{courant}, \text{next})$ est bloquée :
 - Étapes de recherche :
 - Si des sommets se trouvent entre courant et next : Rechercher parmi ceux-ci
 - Sinon rechercher parmi les sommets de next à la fin du tour
 - Si aucun raccourci possible : inverser le sens à la prochaine itération et quitter la boucle
 - iv. Ajouter next au parcours
 - v. Marquer next comme visité et le retirer de non_visités
 - vi. $\text{courant} \leftarrow \text{next}$
 - d. Si aucun sommet n'a été visité pendant l'itération : terminer
 - e. Ajouter le chemin de cette itération à parcours
 - f. Si des sommets restent à visiter :
 - Inverser le sens si nécessaire (selon le flag mis plus tôt)
5. Retourner à s si besoin (et possible), puis renvoyer le parcours complet

Avantages :

- Stratégie simple, efficace si peu de blocages
- Suit un ordre prévisible : moins de recalculs
- Exploite bien l'inégalité triangulaire via les raccourcis

Limites :

- Forte dépendance au tour initial
- Risque de tourner en rond dans un graphe très fragmenté
- Gestion délicate de l'ordre de visite et de l'inversion

3.2 CNN : Christofides + Nearest Neighbor

Principe général

L'algorithme **CNN** utilise deux techniques :

- 1) on suit un tour de Christofides pour visiter un maximum de sommets
- 2) on utilise l'algorithme du plus proche voisin (NN) pour finir

Cela permet d'avoir un parcours assez court tout en s'adaptant aux blocages.

Étapes de l'algorithme :

- Générer un tour de départ avec Christofides
- Parcourir les sommets tant que possible (phase 1)
- Construire un sous-graphe avec les sommets restants
- Appliquer le plus proche voisin sur ce sous-graphe (phase 2)
- Assembler les deux phases pour obtenir le parcours final

L'algorithme :**Entrée :**

G : graphe complet (V, E) avec certaines arêtes bloquées
 s : sommet de départ

Sortie : Un parcours qui passe par tous les sommets malgré les blocages

1. Phase 1 - Suivre Christofides avec gestion des blocages :

- Générer un tour P avec **Christofides**(G, s)
- Initialiser :
 - $\text{non_visités} \leftarrow V \setminus \{s\}$

- `parcours` $\leftarrow [s]$
- `bloquées_connues` $\leftarrow$ ensemble des arêtes bloquées
- c. Parcourir le tour P :
 - Si l'arête suivante est bloquée :
 - Chercher un autre chemin avec Dijkstra
 - Continuer le parcours
 - Sinon, avancer normalement
- 2. **Phase 2 - Explorer les sommets restants :**
 - a. Construire un sous-graphe G' contenant :
 - Les sommets non encore visités U
 - Le sommet de départ s
 - Les arêtes accessibles
 - b. Appliquer `nearest_neighbor` sur G' à partir de s
 - c. Développer les chemins trouvés dans le graphe original
- 3. **Combinaison :**
 - Relier le parcours de la phase 1 et celui de la phase 2
 - Ajouter un retour au départ si besoin

Avantages :

- Bonne capacité d'adaptation aux blocages
- Meilleure couverture que CR si beaucoup de sommets sont bloqués
- Garantit une solution pas trop éloignée de l'optimal

Limites :

- Un peu plus lent que CR
- Dépend beaucoup de la qualité du sous-graphe construit

4 Expérimentation

4.1 Objectifs

L'objectif de cette section est d'évaluer les performances des algorithmes sur différents graphes complets en présence d'arêtes bloquées. Nous cherchons à :

- mesurer la compétitivité (rapport entre le coût du chemin trouvé en ne connaissant pas les arêtes bloquées et le coût optimal en connaissant les blocages),
- observer l'impact du nombre de sommets et du nombre de blocages,
- mesurer le temps d'exécution moyen.

4.2 Protocole de test

Six graphes complets ont été générés aléatoirement avec des poids satisfaisant l'inégalité triangulaire. Les tailles testées sont :

- **16 sommets** (graphes 1 et 2),
- **30 sommets** (graphes 3 et 4),
- **40 sommets** (graphes 5 et 6).

Pour chaque taille, deux versions du graphe sont utilisées :

- une avec **peu de blocages**,
- une avec **beaucoup de blocages**.

Chaque instance est exécutée 30 fois pour estimer un **temps moyen**.

Le coût optimal est estimé en appliquant l'algorithme de Christophides *sur le graphe où les arêtes bloquées ont été supprimées*. Cette approche permet de simuler une solution idéale ayant une connaissance complète des blocages avant de commencer le parcours.

4.3 Résultats de l'algorithme CR

Temps d'exécution moyen (en secondes)

Graphe	Nb sommets	Temps moyen CR
1	16	0.001770
2	16	0.001764
3	30	0.004285
4	30	0.003569
5	40	0.005589
6	40	0.005574

L'algorithme CR est extrêmement rapide, avec un temps d'exécution toujours inférieur à 6 ms même sur des graphes de taille 40.

Coûts et compétitivité

Graphe	Nb sommets	Coût CR	Coût optimal	Ratio CR
1	16	101	81	1.247
2	16	64	46	1.391
3	30	138	142	0.972
4	30	122	88	1.386
5	40	186	172	1.081
6	40	178	118	1.508

Dans la majorité des cas, le ratio de compétitivité est raisonnable, généralement compris entre 1.1 et 1.5.

4.4 Analyse

- **Temps d'exécution** : CR est très rapide, même sur des graphes avec beaucoup de sommets.
- **Performances** : CR s'en sort bien lorsqu'il y a peu de blocages, mais peut dégrader la qualité du parcours lorsque de nombreuses arêtes sont bloquées.
- **Cas extrêmes** : le comportement peut être surprenant dans certains cas favorables où les raccourcis naturels suffisent à contourner les blocages.

L'approche basée sur Christophides et des raccourcis locaux montre une bonne efficacité globale, mais pourrait bénéficier d'une stratégie plus globale lorsque les blocages sont trop nombreux.

4.5 Résultats de l'algorithme CNN

Temps d'exécution moyen (en secondes)

Graphe	Nb sommets	Temps moyen CNN
1	16	0.000863
2	16	0.000600
3	30	0.001422
4	30	0.001547
5	40	0.002326
6	40	0.002283

CNN montre des temps d'exécution légèrement plus élevés que CR, mais reste très rapide avec des temps inférieurs à 2.5 ms même pour les graphes de 40 sommets.

Coûts et compétitivité

Graphe	Nb sommets	Coût CNN	Coût optimal	Ratio CNN
1	16	83	68	1.221
2	16	64	46	1.391
3	30	138	142	0.972
4	30	122	88	1.386
5	40	186	172	1.081
6	40	178	118	1.508

CNN obtient des ratios de compétitivité comparables à CR, avec une légère amélioration dans certains cas (comme le graphe 1 où le ratio passe de 1.247 à 1.221).

4.6 Analyse comparative

- **Vitesse** : CNN est en moyenne 1.2x plus lent que CR, ce qui reste tout à fait acceptable
- **Qualité des solutions** : Les deux algorithmes obtiennent des résultats très similaires sur nos instances de test
- **Robustesse** : CNN montre une légère amélioration dans les cas avec peu de blocages (graphe 1)
- **Compétitivité** : Les ratios sont très proches, confirmant la similarité des approches

Observation clé : Bien que CNN soit théoriquement meilleur grâce au deux phases : Christofides+NN, sur nos instances de test les deux algorithmes montrent des performances similaires.

5 Conclusion

À travers nos expérimentations, nous avons comparé les algorithmes **CR** et **CNN** pour résoudre le problème du voyageur canadien couvrant (CCTP).

L'algorithme **CNN** obtient généralement de meilleurs itinéraires que **CR** : le parcours final est plus court, ce que l'on voit sur le rapport de compétitivité. Cela s'explique par la deuxième phase de CNN qui utilise le plus proche voisin sur un sous-graphe construit à partir des informations apprises pendant l'exploration. Cette étape permet d'adapter intelligemment le parcours aux blocages détectés.

Cependant, **CNN est un peu plus lent** que **CR**. La raison principale est la construction du sous-graphe G' et l'application de l'algorithme du plus proche voisin, qui ajoutent un coût de calcul supplémentaire après la première phase de Christofides. De plus, la recherche des plus courts chemins intermédiaires (via Dijkstra) ralentit légèrement l'algorithme.